# MLOps Lab Assignment 2

Tracking a Pre-trained Sentiment Model with MLflow

| Item | Details |
|---|---|
| **Student** | Aishwarya S |
| **USN** | 4GW23CI003 |
| **Tools** | Python, MLflow 3.1.4, Hugging Face Transformers, PyTorch, pandas |
| **MLflow experiment** | sentiment-inference-lab |

## 1. Objective

This lab uses MLflow to track inference runs of a ready-made Hugging Face sentiment model. The same model is run on the same 10 labelled sentences with different batch sizes. Parameters, metrics and an artifact are logged for every run, the runs are compared in the MLflow UI, and the better run is reported. No model training is involved.

## 2. Setup

Everything was run in Windows Command Prompt inside a virtual environment named mlops-env, as the (mlops-env). The steps were:

- created and activated the environment (python -m venv mlops-env, then mlops-env\Scripts\activate)
- upgraded pip and install the packages with pip install mlflow transformers torch pandas

verify the installation, and pre-download the model once so later runs use the cached copy. The terminal output "Device set to use cpu" confirms that inference ran on the CPU.

## 3. Task 1: Running the Tracked Experiment

mlflow_lab.py defines run_experiment(model_name, batch_size, run_name), which loads the model, opens an MLflow run and logs the following:

| Type | Logged items |
|---|---|
| Parameters | model_name, batch_size, num_samples |
| Metrics | accuracy, avg_confidence, total_latency_sec (time of the inference call only) |
| Artifact | predictions.csv with the text, actual label, predicted label and confidence of each sentence |

The dataset has 5 positive and 5 negative sentences. The script is shown below; its last line is the third run added in Task 3.

```
import time
import mlflow
import pandas as pd
from transformers import pipeline

# Small labelled dataset (10 sentences)
data = [
    ("I loved this movie, it was fantastic!", "POSITIVE"),
    ("The service was terrible and slow.", "NEGATIVE"),
    ("What a wonderful experience.", "POSITIVE"),
    ("I regret buying this product.", "NEGATIVE"),
    ("The food was delicious and fresh.", "POSITIVE"),
    ("This is the worst app I have ever used.", "NEGATIVE"),
    ("Absolutely brilliant performance by the team.", "POSITIVE"),
    ("The battery dies within an hour.", "NEGATIVE"),
    ("Great value for money, highly recommend.", "POSITIVE"),
    ("I am very disappointed with the quality.", "NEGATIVE")
]

texts = [t for t, _ in data]
labels = [l for _, l in data]
```

```
mlflow.set_experiment("sentiment-inference-lab")


def run_experiment(model_name, batch_size, run_name):

    clf = pipeline("sentiment-analysis", model=model_name)

    with mlflow.start_run(run_name=run_name):

        # Parameters
        mlflow.log_param("model_name", model_name)
        mlflow.log_param("batch_size", batch_size)
        mlflow.log_param("num_samples", len(texts))

        # Inference
        start = time.time()
        preds = clf(texts, batch_size=batch_size)
        latency = time.time() - start

        pred_labels = [p["label"] for p in preds]

        accuracy = sum(
            p == l for p, l in zip(pred_labels, labels)
        ) / len(labels)

        avg_conf = sum(p["score"] for p in preds) / len(preds)

        # Metrics
        mlflow.log_metric("accuracy", accuracy)
        mlflow.log_metric("avg_confidence", avg_conf)
        mlflow.log_metric("total_latency_sec", latency)

        # Artifact
        df = pd.DataFrame({
            "text": texts,
            "actual": labels,
            "predicted": pred_labels,
            "confidence": [p["score"] for p in preds]
        })

        df.to_csv("predictions.csv", index=False)
        mlflow.log_artifact("predictions.csv")

        print(
            run_name,
            "-> accuracy:", accuracy,
            "latency:", round(latency, 3)
        )


MODEL = "distilbert-base-uncased-finetuned-sst-2-english"

run_experiment(MODEL, batch_size=1, run_name="run_batch1")
run_experiment(MODEL, batch_size=8, run_name="run_batch8")
run_experiment(MODEL, batch_size=4, run_name="run_batch4")
```

mlflow_lab.py

The script was run with python mlflow_lab.py. Figure 1 shows the first execution (experiment created, run_batch1 and run_batch8 printed), the start of mlflow ui at http://127.0.0.1:5000.

```
(mlops-env) C:\Users\Aishwarya S>python mlflow_lab.py
2026/10/31 20:47:05 INFO mlflow.tracking.fluent: Experiment with name 'sentiment-inference-lab' does not exist. Creating a new experiment.
Device set to use cpu
run_batch1 -> accuracy: 1.0 latency: 0.475
Device set to use cpu
run_batch8 -> accuracy: 1.0 latency: 0.225

(mlops-env) C:\Users\Aishwarya S>mlflow ui
INFO:waitress:Serving on http://127.0.0.1:5000
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 2
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 2
WARNING:waitress.queue:Task queue depth is 2
WARNING:waitress.queue:Task queue depth is 2
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 2
WARNING:waitress.queue:Task queue depth is 1
WARNING:waitress.queue:Task queue depth is 2
WARNING:waitress.queue:Task queue depth is 1

Aborted!

(mlops-env) C:\Users\Aishwarya S>python mlflow_lab.py
Device set to use cpu
run_batch1 -> accuracy: 1.0 latency: 0.94
Device set to use cpu
run_batch8 -> accuracy: 1.0 latency: 0.259
Device set to use cpu
run_batch4 -> accuracy: 1.0 latency: 0.205

(mlops-env) C:\Users\Aishwarya S>mlflow ui
INFO:waitress:Serving on http://127.0.0.1:5000
```

**Figure 1:** *Terminal output of python mlflow_lab.py*

## 4. Task 2: Exploring the MLflow UI

The experiment sentiment-inference-lab was opened at http://127.0.0.1:5000. Figure 2 shows the runs table with the two runs of Task 1, both finished, created by mlflow_lab.py.

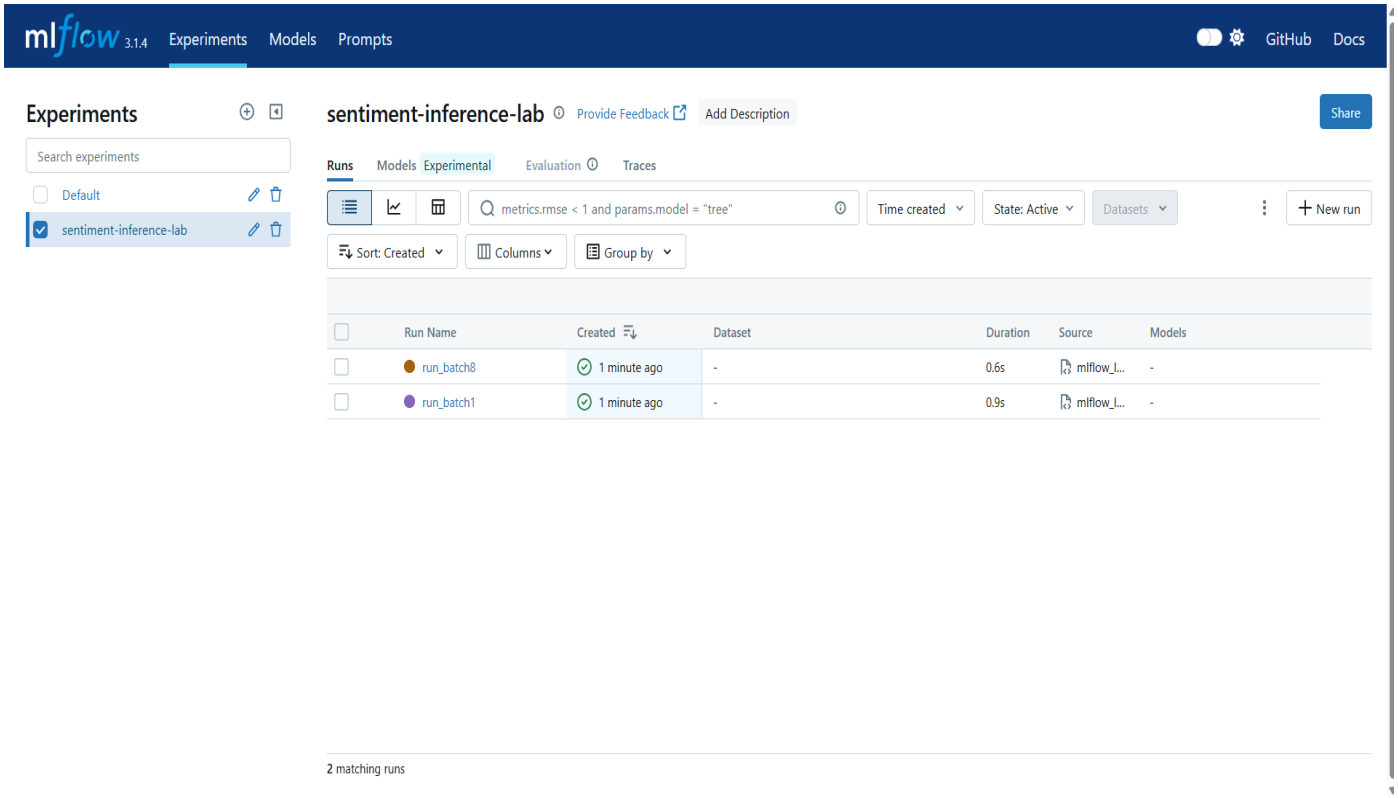



**Figure 2:** *Runs table of sentiment-inference-lab showing run_batch8 and run_batch1.*

Both runs were then selected and compared. In the parallel coordinates plot (Figure 3), batch sizes 8 and 1 both lead to an accuracy of 1.0. The details table (Figure 4) highlights batch_size as the only differing parameter, and the metrics show identical accuracy and confidence but a latency of 0.225 s for run_batch8 against 0.475 s for run_batch1.

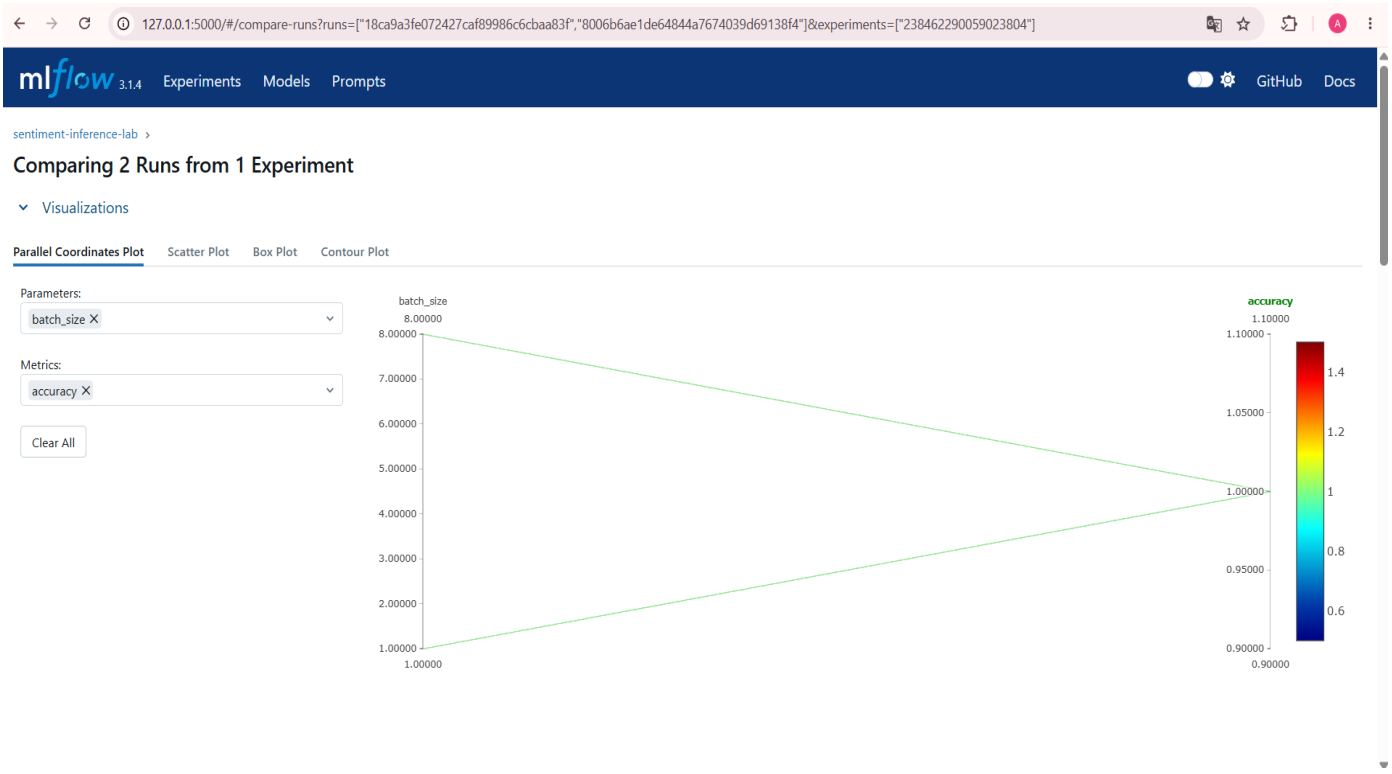



**Figure 3:** *Compare view: parallel coordinates plot of batch_size against accuracy.*

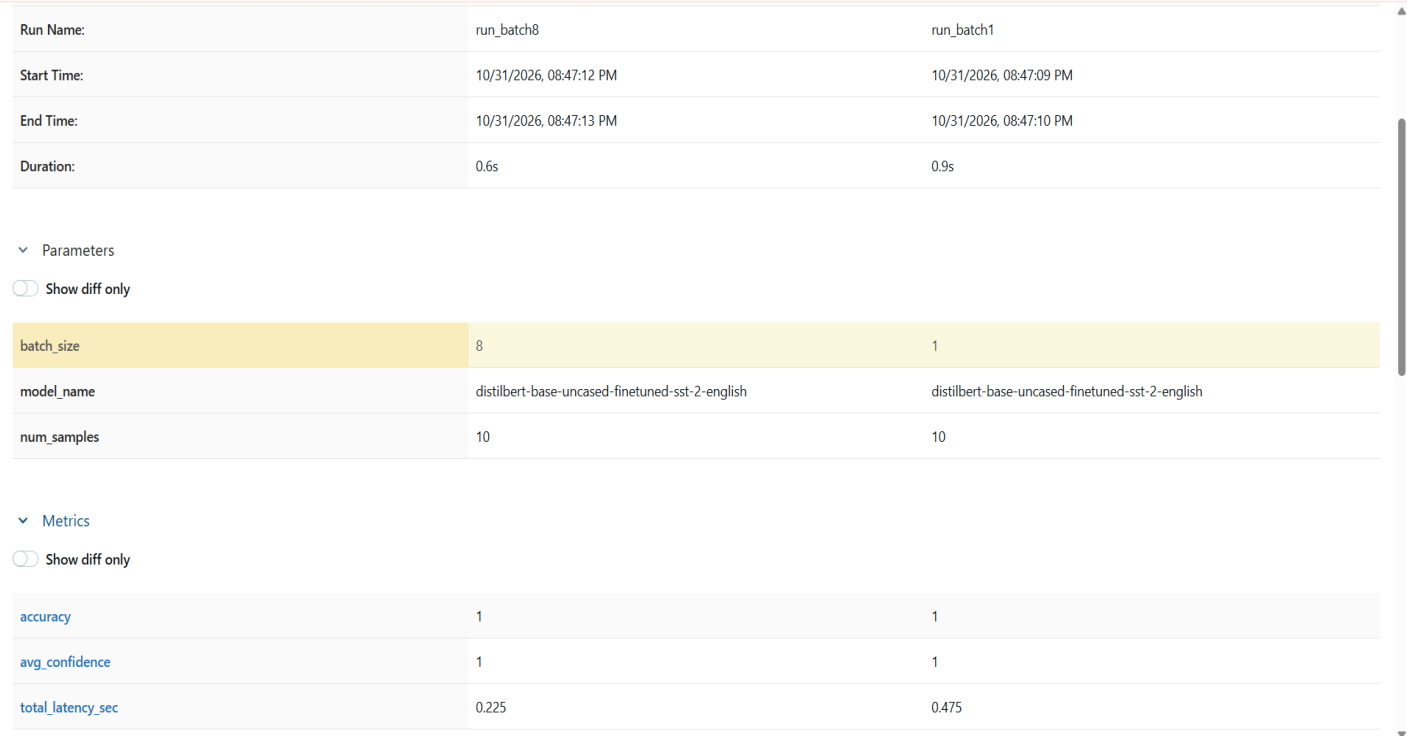



**Figure 4:** *Compare view: run details, parameters and metrics of run_batch8 and run_batch1.*

Individual runs were opened as well. Figures 5 to 8 show the Overview and Model metrics tabs of run_batch1 and run_batch8 (latest copies, logged by the second execution). The predictions.csv file of each run is stored under its Artifacts tab.

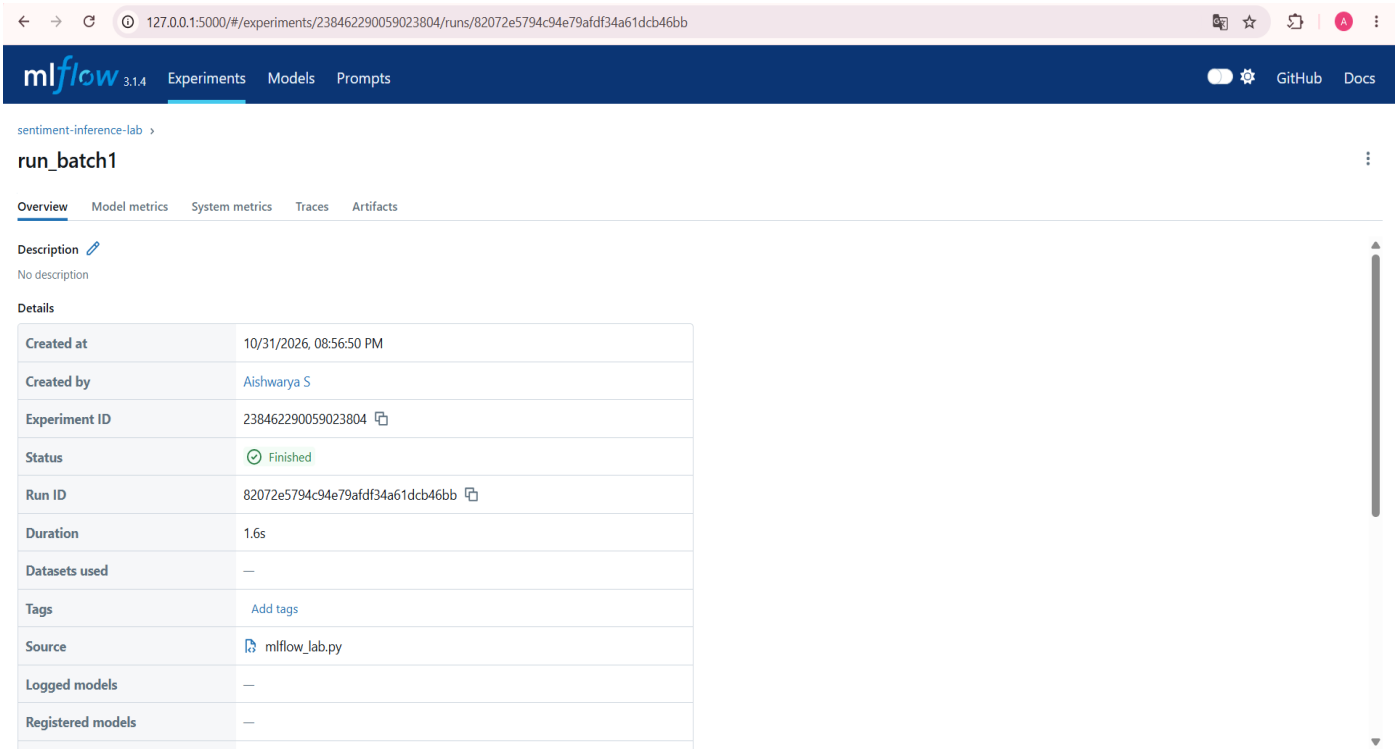



**Figure 5:** *run_batch1: Overview tab*

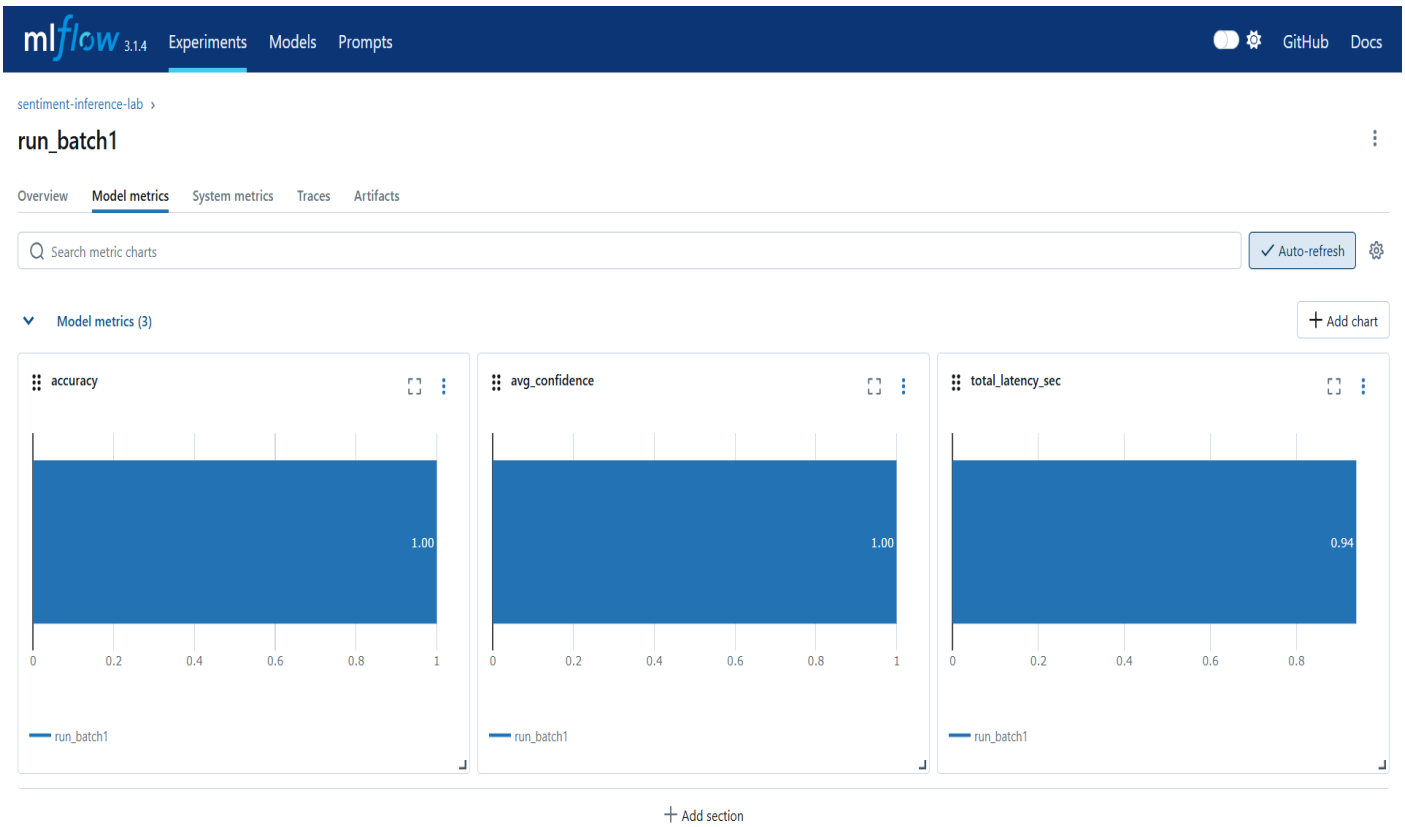



**Figure 6:** *run_batch1: Model metrics.*

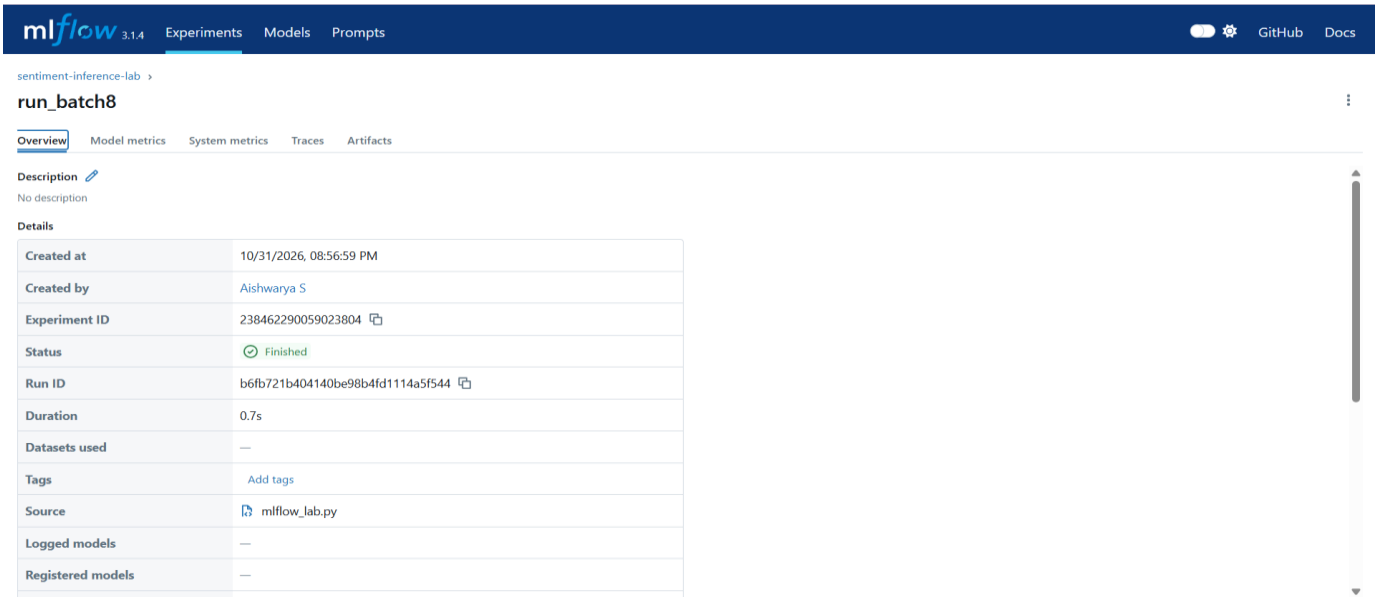



**Figure 7:** *run_batch8: Overview tab*

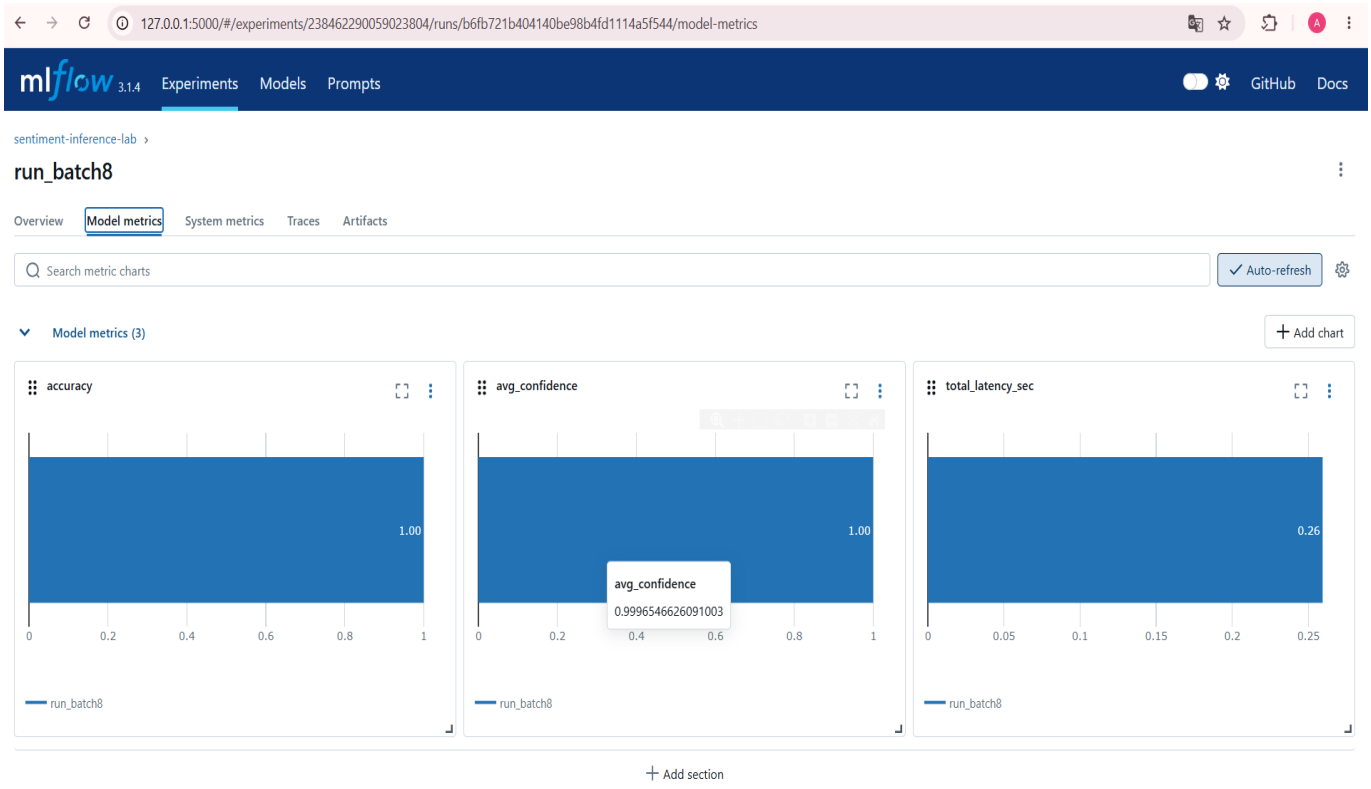



**Figure 8:** *run_batch8: Model metrics*

## 5. Task 3: Running a Third Experiment

The same model was run with batch_size=4 by adding run_experiment(MODEL, batch_size=4, run_name="run_batch4") at the bottom of the script. Re-running the script executes the whole file, so run_batch1 and run_batch8 were logged again too. The new run_batch4 appears in the runs table (Figure 9).

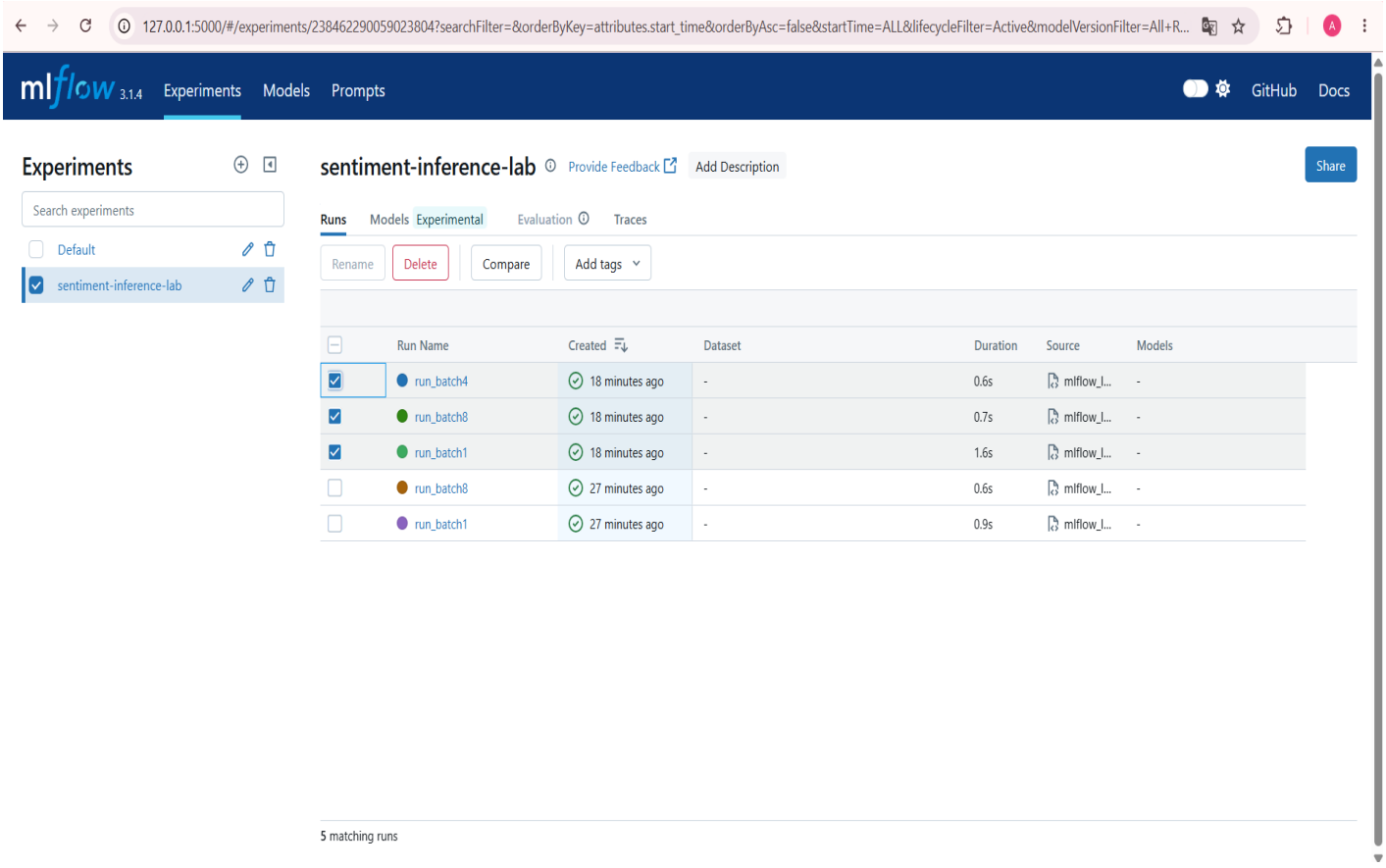



**Figure 9:** *Runs table with five runs; the three runs of the second execution are selected for Compare.*

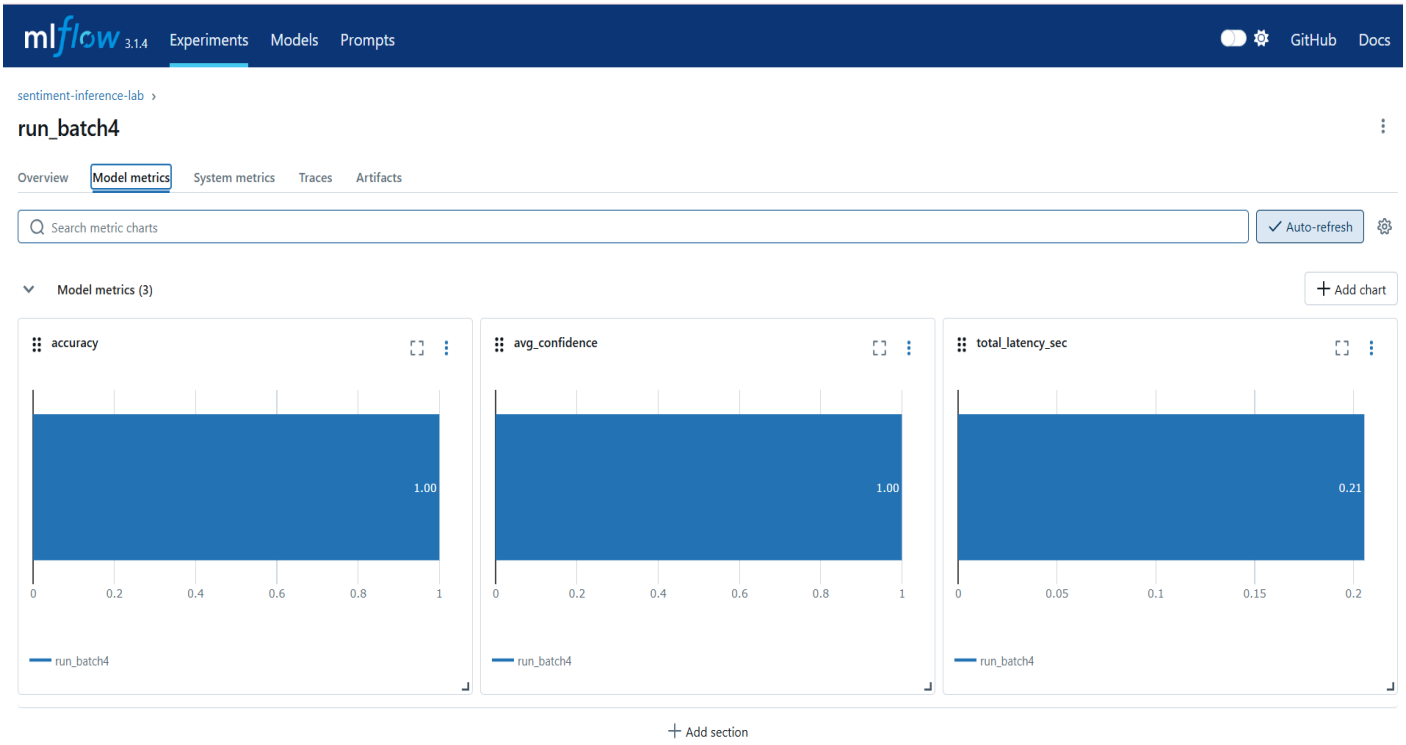



**Figure 10:** *run_batch4: Model metrics (accuracy 1.00, avg_confidence 1.00, total_latency_sec 0.21).*

The three newest runs were compared. In the parallel coordinates plot (Figure 11) all lines meet at the same accuracy and confidence values and separate only at latency, where batch size 1 is at the slow (red) end and batch sizes 4 and 8 are at the fast (blue) end. In the details table (Figure 12) only total_latency_sec differs between the runs.

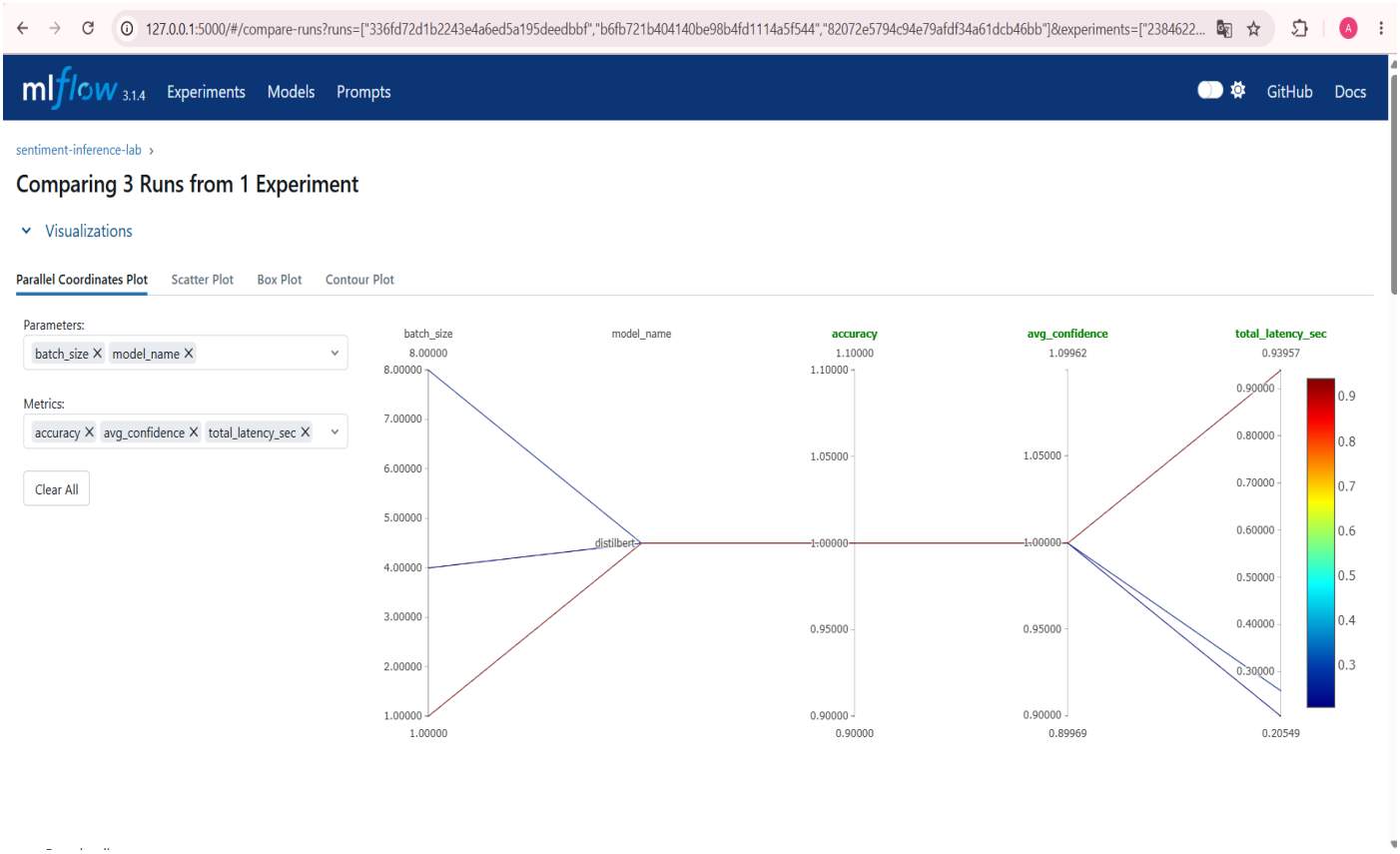



**Figure 11:** *Compare view of three runs: parallel coordinates plot.*

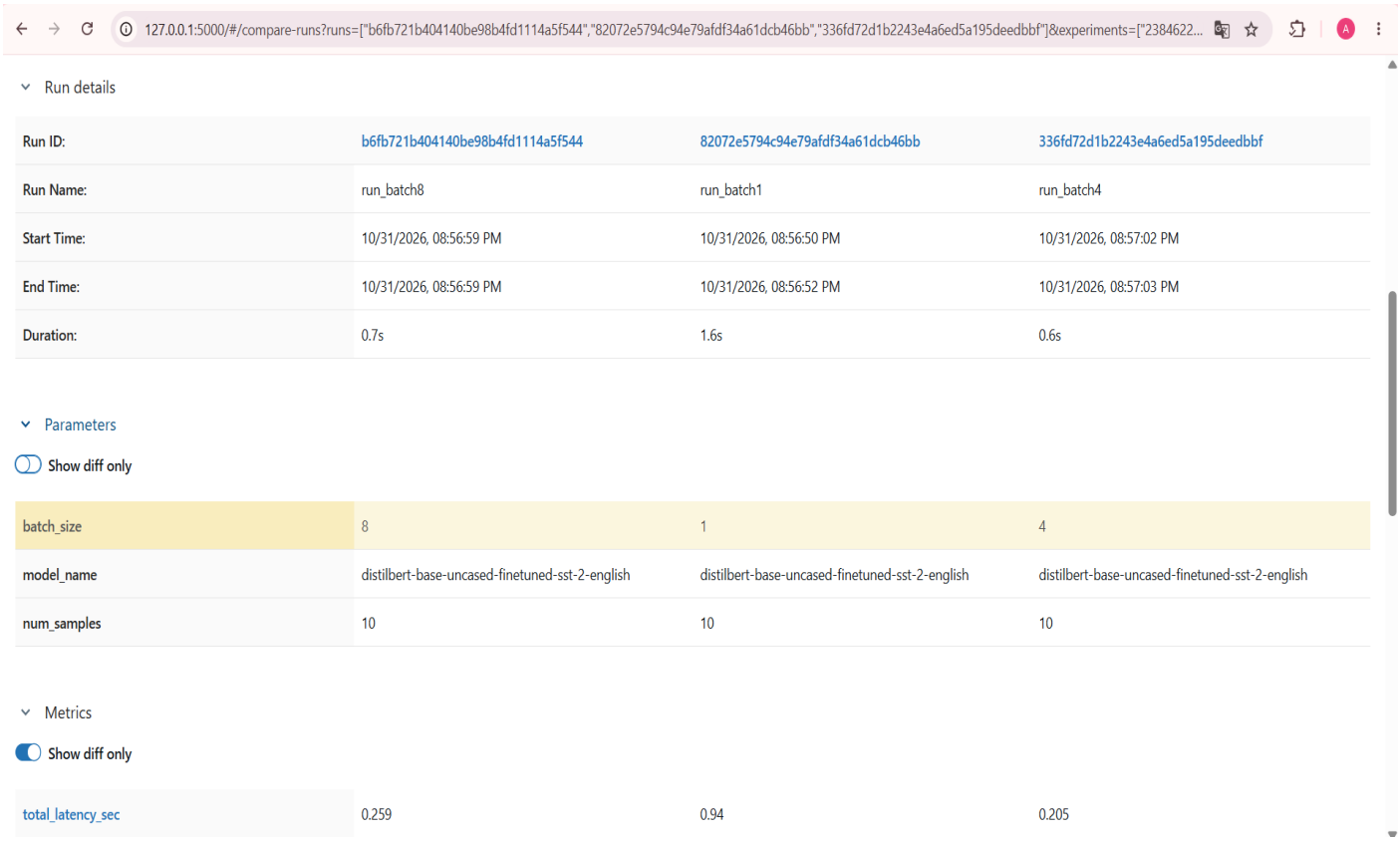



**Figure 12:** *Compare view of three runs: run details, parameters and differing metrics.*

## 6. Results

| Run | Execution | Batch size | Accuracy | Avg. confidence | Latency (s) | Duration |
|---|---|---|---|---|---|---|
| run_batch1 | 1st (Task 1) | 1 | 1.0 | about 1.00 | 0.475 | 0.9 s |
| run_batch8 | 1st (Task 1) | 8 | 1.0 | about 1.00 | 0.225 | 0.6 s |
| run_batch1 | 2nd (Task 3) | 1 | 1.0 | about 1.00 | 0.940 | 1.6 s |
| run_batch8 | 2nd (Task 3) | 8 | 1.0 | 0.99965 | 0.259 | 0.7 s |
| run_batch4 | 2nd (Task 3) | 4 | 1.0 | about 1.00 | 0.205 | 0.6 s |

*Table 1: Results of all runs. The best run (lowest latency at full accuracy) is highlighted.*

## 7. Task 4: (Answers to the Questions)

- **Q1. Lowest latency:** run_batch4 had the lowest latency (0.205 s); in the first execution it was run_batch8 (0.225 s against 0.475 s for run_batch1).

  **Why:** larger batches send several sentences through the model in one forward pass, which cuts per-call overhead and uses the CPU more efficiently than ten separate batch-size-1 calls.
- **Q2. Accuracy did not change**. It stayed at 1.0 (10/10) in every run, because batch size only changes how the inputs are grouped, not the model or its predictions, and the 10 sentences are easy to classify**.**
- **Q3. Benefits :**
  - **Benefit 1 of MLflow:** parameters, metrics and artifacts of every run are stored permanently with a run ID and timestamp, whereas terminal output is lost once the window is closed.
  - **Benefit 2 of MLflow:** runs can be compared side by side with tables and charts in the UI instead of reading and comparing printed numbers by hand.

## 8. Conclusion

- Three runs of a pre-trained DistilBERT sentiment model were tracked and compared with MLflow.
- Accuracy was 100% for every batch size, while batching (sizes 4 and 8) was clearly faster than batch size 1, with run_batch4 the fastest run at 0.205 s.